Methodology: ML-Based Cooling Technique Recommendation System

1. System Overview

This system provides intelligent, data-driven recommendations for optimal data center cooling technique selection. The pipeline integrates supervised machine learning, physics-based simulation, climate-aware estimation, and large language model (LLM) explanation into a unified recommendation engine accessible through a React-based frontend. The three candidate techniques evaluated are Air Side Economization, Evaporative Cooling, and Chilled Water Cooling [25].

2. Dataset Generation

2.1 Synthetic Scenario Sampling

Training data was generated through Monte Carlo sampling [26] across six input features representing the operating environment of a data center: ambient dry-bulb outdoor temperature (tempC, °C), relative humidity (rh, %), IT equipment thermal load (itLoadKW, kW), grid electricity tariff (electricityPrice, USD/kWh), water supply tariff (waterPrice, USD/litre), and grid carbon intensity (carbonFactor, kgCO2/kWh). A total of 500 scenarios were sampled uniformly across realistic operational ranges, covering diverse climates, load profiles, and economic contexts to ensure the model generalises across geographies and tariff structures [27].

2.2 Multi-Technique Simulation

For each sampled scenario, identical input parameters were submitted concurrently to all three cooling system simulators: the Air Side Economization backend, the Evaporative Cooling backend, and the Chilled Water Spring Boot service. Each simulator returned annual energy consumption, operating cost, water usage, and carbon emissions for that scenario. Using identical inputs across all three techniques ensures a controlled, fair comparison where the only variable is the cooling method itself [28].

2.3 Feasibility-Gated Labeling

Labels were assigned using a two-stage deterministic rule. First, physical feasibility constraints were applied based on established thermodynamic operating limits [29]:

- Air Side Economization: feasible only if tempC ≤ 27°C and rh ≤ 80%
- Evaporative Cooling: feasible only if rh ≤ 70% and tempC ≤ 45°C
- Chilled Water: always feasible (mechanical refrigeration system)

From the feasible set, the technique with minimum energy consumption was selected as the label bestTechnique. This deterministic labeling rule produces consistent, reproducible training targets from simulation outputs without manual annotation [30].

Formally: bestTechnique(i) = argmin E(i,t) for t in feasible set F(i)

2.4 Class Distribution and Imbalance

The resulting dataset contains 500 labeled rows: Evaporative Cooling (43%), Chilled Water (43%), Air Side Economization (14%). The underrepresentation of Air Economizer reflects its narrow feasibility window. Class imbalance was addressed using class_weight='balanced' in the Random Forest, which adjusts weights inversely proportional to class frequencies [31].

3. Machine Learning Model

3.1 Algorithm Selection

A Random Forest Classifier [32] was selected for its ability to model non-linear decision boundaries between cooling regimes, robustness to feature scale differences, support for class imbalance, and interpretability through Gini-based feature importance scores. The ensemble of decision trees uses bootstrap aggregation and random feature subsampling, producing a final prediction by majority vote [33].

3.2 Preprocessing Pipeline

```
Input (6 features)
↓
SimpleImputer (strategy = median)
↓
StandardScaler (zero mean, unit variance)
↓
RandomForestClassifier (class_weight = balanced)
```

3.3 Hyperparameter Optimisation

Hyperparameter search was conducted using RandomizedSearchCV [34] over 20 iterations with 5-fold Stratified K-Fold cross-validation, optimising Macro-F1. Search space: n_estimators in {300, 500, 700, 900}, max_depth in {None, 10, 16, 24, 32}, min_samples_split in {2, 4, 6, 10}, min_samples_leaf in {1, 2, 4}, max_features in {sqrt, log2, None}. Dataset split 80/20 with stratification and fixed random seed 42.

3.4 Evaluation

Performance was assessed on the held-out 20% test set using accuracy, Macro-F1, per-class precision/recall/F1, and confusion matrix. Robustness was validated across 30 repeated stratified shuffle splits reporting mean ± standard deviation and 95% confidence intervals. The trained pipeline was serialised to cooling_recommender_rf.pkl.

4. Inference Pipeline

4.1 Input Collection from Database

When a user completes any simulation, the system retrieves from Supabase: real annual cost, emissions, water, energy, PUE, 8,760 hourly rows of temperature/humidity/IT load, and technique-specific findings (airflow violations, hotspot rack counts, cooling adequacy status, Phase 4 compliance gate results).

4.2 Scenario Derivation

Annual averages are computed from the 8,760 hourly rows: T_avg = (1/8760) * sum(T_h), H_avg = (1/8760) * sum(H_h), L_avg = (1/8760) * sum(L_h). These averages combined with the user's configured tariff and carbon settings form the six-feature vector submitted to the ML model.

4.3 Climate-Aware Technique Estimation

Since only one technique has been simulated, the other two are estimated using climate-aware dynamic efficiency factors. This is valid because all three techniques would operate at the same physical location facing identical outdoor conditions [35]. Three penalty variables are derived from real site conditions:

P_temp = clamp((T_avg - 18) / 20, 0, 1) [temperature penalty for Air Economizer]

P_rh_air = clamp((H_avg - 40) / 50, 0, 1) [humidity penalty for Air Economizer]

P_rh_evap = clamp((H_avg - 20) / 60, 0, 1) [humidity penalty for Evaporative]

Dynamic cost efficiency factors:

f_cost(Air) = 0.30 + 0.55 * (0.7 * P_temp + 0.3 * P_rh_air)

f_cost(Evap) = 0.45 + 0.40 * P_rh_evap

f_cost(CW) = 1.0 - clamp((L_avg - 200) / 2000, 0, 0.1)

The current technique's real cost is used to back-calculate a common baseline: baseline = C_real / f_current. Each other technique's cost is then estimated as: C_target = baseline * f_target. The same procedure applies to emissions, water, and energy. The current technique always retains its real database values unchanged.

4.4 Feasibility Assessment

Each technique is assessed against physical thresholds at the site's average conditions. Each violated condition increments a violation counter. Techniques with one or more violations are marked infeasible and excluded from the primary recommendation.

4.5 ML Prediction and Fallback Decision

The six-feature scenario vector is passed to the loaded Random Forest pipeline. If the predicted technique is feasible it becomes the final recommendation. If infeasible, the system selects the feasible technique with the lowest composite weighted score:

S_i = 0.5 * C_norm_i + 0.3 * E_norm_i + 0.2 * W_norm_i

where C_norm, E_norm, W_norm are min-max normalised cost, emissions, and water values across the three techniques. Weights are configurable via environment variables. This multi-criteria scoring follows established weighted sum decision-making methodology [27].

4.6 LLM-Generated Justification and Future Impact

Numerical evidence — real metrics, failure findings, estimated comparisons, and site conditions — is assembled into a structured prompt submitted to the Groq inference API using the LLaMA 3.3 70B model [28]. A fallback chain of smaller models is used if the primary model is rate-limited. The LLM generates 8-10 professional sentences structured to champion the recommended technique, resolving every tradeoff with exact numbers. Separately, Year 1, 3, and 5 cost/emissions/water projections are submitted to the same model, which generates a forward-looking narrative. Five-year savings: Savings_5yr = (C_alt - C_rec) * 5.

4.7 Frontend Integration

The recommendation pipeline is triggered automatically upon simulation completion in store.ts, which calls getMLRecommendation() from simulationIntelligenceService.ts. The response — containing model_recommendation, why_this_is_recommended, future_impact_paragraph, and comparison_table — is stored in Supabase and rendered in the React frontend across the Recommendations tab (SimulationRecommendations.tsx), simulation charts, PDF export, and user profile report.

References

[25] R. E. Caflisch, "Monte Carlo and quasi-Monte Carlo methods," Acta Numerica, vol. 7, pp. 1-49, 1998.

[26] C. L. Hwang and A. S. M. Masud, Multiple Objective Decision Making — Methods and Applications. Berlin: Springer, 1979.

[27] Meta AI, LLaMA 3: Open Foundation and Fine-Tuned Chat Models, 2024. [Online]. Available: https://ai.meta.com/llama/

[28] ASHRAE, Thermal Guidelines for Data Processing Environments, 4th ed. Atlanta: ASHRAE, 2015.

[29] T. Mitchell, Machine Learning. New York: McGraw-Hill, 1997.

[30] N. V. Chawla et al., "SMOTE: Synthetic minority over-sampling technique," Journal of Artificial Intelligence Research, vol. 16, pp. 321-357, 2002.

[31] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.

[32] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.

[33] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[34] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," Journal of Machine Learning Research, vol. 13, pp. 281-305, 2012.

[35] K. Ebrahimi, G. F. Jones, and A. S. Fleischer, "A review of data center cooling technology, operating conditions and the corresponding low-grade waste heat recovery opportunities," Renewable and Sustainable Energy Reviews, vol. 31, pp. 622-638, 2014.